SAP Application Development (S/4 HANA)

OData Services

Motivation
This chapter will show you how to develop OData services and consume them in an SAPUI5 application. Therefore, you will learn how ABAP can be used to implement services, and how SAPUI5 can be used to develop web interfaces which consume these services.

Prerequisites
Before starting this exercise, you should have fundamental ABAP programming skills and be familiar with the Eclipse IDE. Furthermore, it is recommended that you have completed the *HTML5, CSS & JavaScript* and *SAPUI5* chapters. The completion of the chapter *ABAP Objects* is also helpful.

Content
This chapter explains the complete lifecycle of an OData service. First, a new project for the implementation of an OData service will be created. Second, the service will be implemented, followed by the activation and test of the service. Afterwards, an SAPUI5 application is developed which consumes the service. For this, first a view is created, which will show the data. Second, a controller is implemented which calls the OData service. Finally, the application will be deployed to the SAP backend.

Lecture notes
Students can go on with their accounts from the previous chapters mentioned in the prerequisites. If you skipped these chapters, your students may need to enter a developer key when creating their services. Please consult the lecture notes for further assistance on this topic. Furthermore, students should always use an individual three-digit number to mark their repository objects. In this exercise, these are displayed as ### and have to be replaced by the students' individual numbers. In addition, please replace $$ with the two-digit institution id.

- **Product**: All
- **Level**: Advanced
- **Focus**: Programming
- **Version**: 2.0
- **Author**: UCC Technical University of Munich

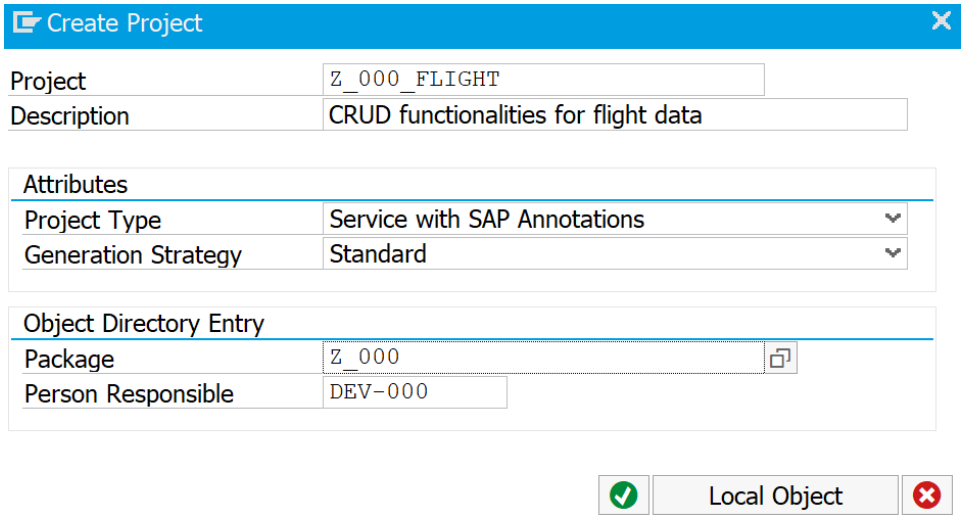
Task 1: Define a new Project and generate Objects for an OData Service

Short description: Using the integrated SAP GUI to create a new project in the Gateway Service Builder and generate the necessary runtime objects

Start Eclipse, and open the ABAP perspective. Open the integrated SAP GUI in Eclipse by clicking on , selecting an existing ABAP project and clicking **OK**. If you currently have no existing ABAP project, please refer to the *Introduction to SAP Programming* chapter on how you create a new ABAP project.

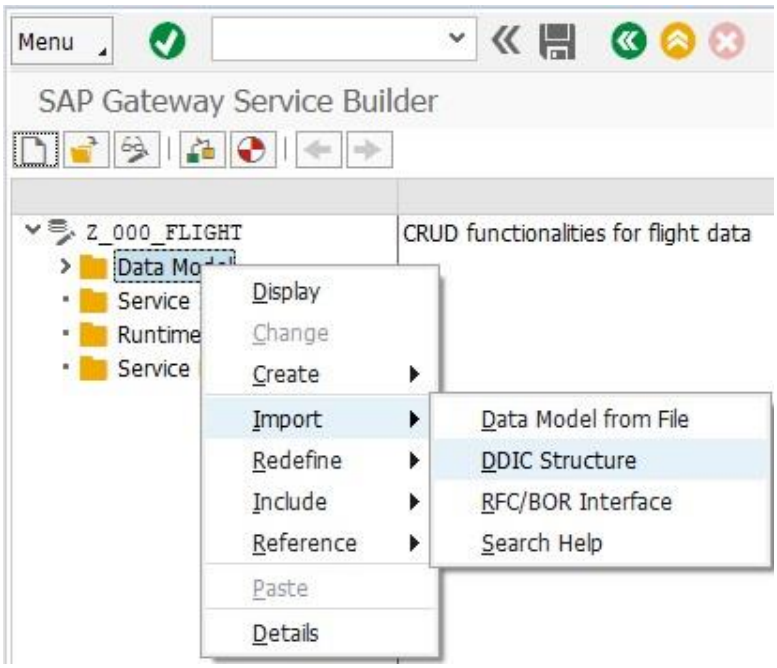
Open Gateway Service Builder

In the integrated SAP GUI, call transaction code **SEGW**, which opens the Gateway Service Builder. In the upper left of the integrated SAP GUI, click on **Create Project** . In the appearing popup, enter Project **Z_$$_###_FLIGHT**, a description for your project, and select your package **Z_$$_###,** which you should have already used in the previous exercises (Please refer to the *Development of a first application* exercise if you do not have this package yet and need to create it). Click on **Enter** to create your project.



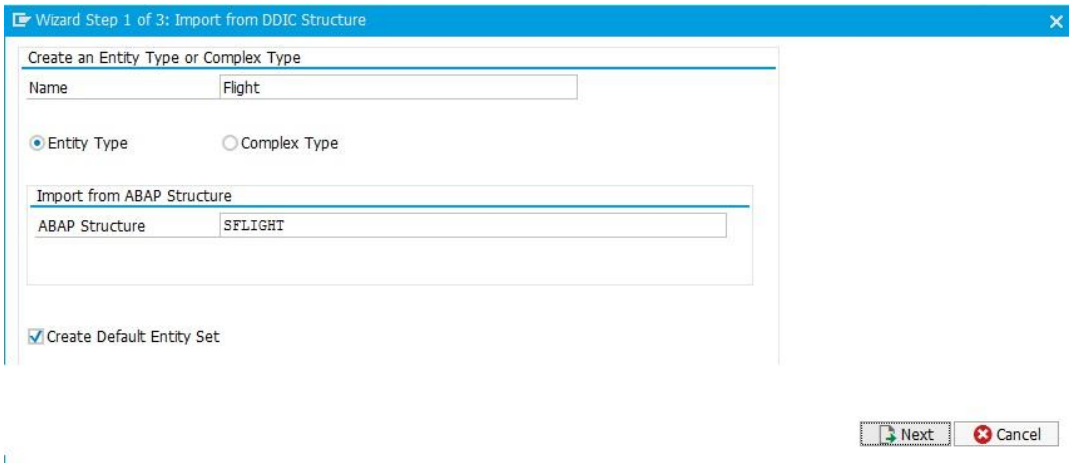
Create a new project

Next, the data model which serves as basis for the OData service has to be defined. For this, in the context menu of your created service, right-click on **Data Model**, and select **Import->DDIC Structure**.



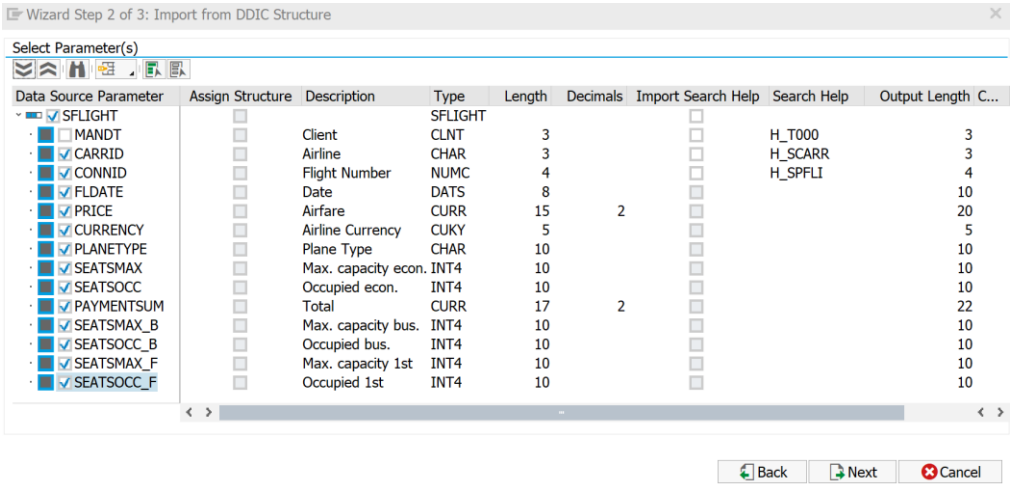
Import DDIC Structure

A popup appears. Enter **Flight** as Name, and select **SFLIGHT** as ABAP structure, from which you want to import the data. In this case, **SFLIGHT** is a table in the SAP system which contains flight data. Click on **Next**.



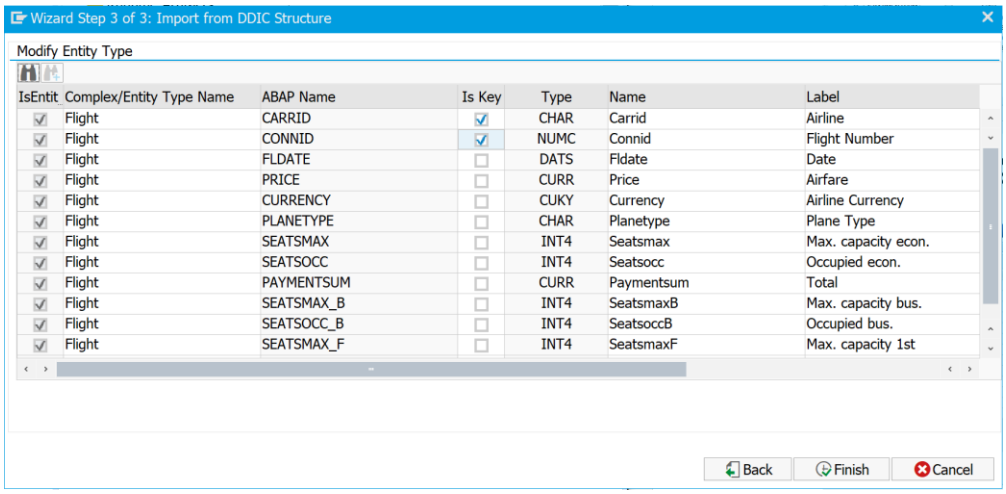
Select all fields except **MANDT**. These data fields will be contained in the OData service later. Click **Next.**

Select fields



Finally, the key fields have to be defined. These will be used to make every data entry uniquely identifiable. Choose the **CARRID** and **CONNID** field and check them in the **Is Key** column.

Determine key elements



Click on **Finish**. Go to the folder **Data Model->Entity Types->Flight** and double-click on **Properties**. Please mark the fields **Fldate, SeatsmaxB, SeatsoccB, SeatsmaxF, and SeatsoccF** as **Nullable**.

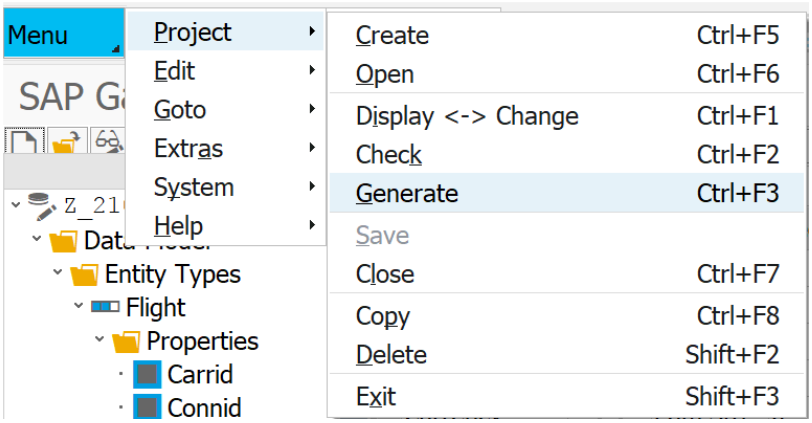
Properties

Name	Is Key	Edm Type	Prec.	Scale	Max L	Unit Prop.	Creat	Updat	Sortab	Nullab	Filt.	Label	La	Comp. Ty	Field	AB
Carrid	☑	Edm.Stri	0	0	3		☐	☐	☐	☐	☐	Airline	T		CARRID	
Connid	☑	Edm.Stri	0	0	4		☐	☐	☐	☐	☐	Flight Nu	T		CONNID	
Fldate	☐	Edm.Dat	7	0	0		☐	☐	☐	☑	☐	Date	T		FLDATE	
Price	☐	Edm.Deci	16	3	0		☐	☐	☐	☐	☐	Airfare	T		PRICE	
Currency	☐	Edm.Stri	0	0	5		☐	☐	☐	☐	☐	Airline Cu	T		CURREN	
Planetype	☐	Edm.Stri	0	0	10		☐	☐	☐	☐	☐	Plane Ty	T		PLANETY	
Seatsmax	☐	Edm.Int3	0	0	0		☐	☐	☐	☐	☐	Max. cap	T		SEATSMA	
Seatsocc	☐	Edm.Int3	0	0	0		☐	☐	☐	☐	☐	Occupied	T		SEATSOC	
Paymentsum	☐	Edm.Deci	18	3	0		☐	☐	☐	☐	☐	Total	T		PAYMEN	
SeatsmaxB	☐	Edm.Int3	0	0	0		☐	☐	☐	☑	☐	Max. cap	T		SEATSMA	
SeatsoccB	☐	Edm.Int3	0	0	0		☐	☐	☐	☑	☐	Occupied	T		SEATSOC	
SeatsmaxF	☐	Edm.Int3	0	0	0		☐	☐	☐	☑	☐	Max. cap	T		SEATSMA	
SeatsoccF	☐	Edm.Int3	0	0	0		☐	☐	☐	☑	☐	Occupied	T		SEATSOC	

Make fields nullable

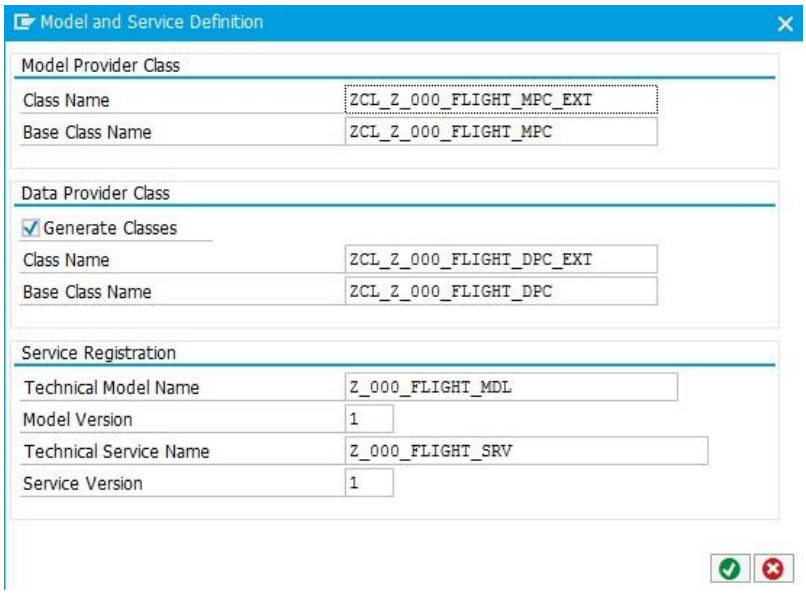
Save your data model by clicking on  in the integrated SAP GUI. Choose a transport request to complete the saving process. Refer to the *Development of a first program* exercise if you do not have a transport request yet.

In the next step, the service elements have to be created based on the defined data model. For this purpose, go to the menu bar and choose **Project->Generate**.



Generate service elements

Make sure that your three-digit user number is part of the class and data provider names. Leave all other names and selections as they are.



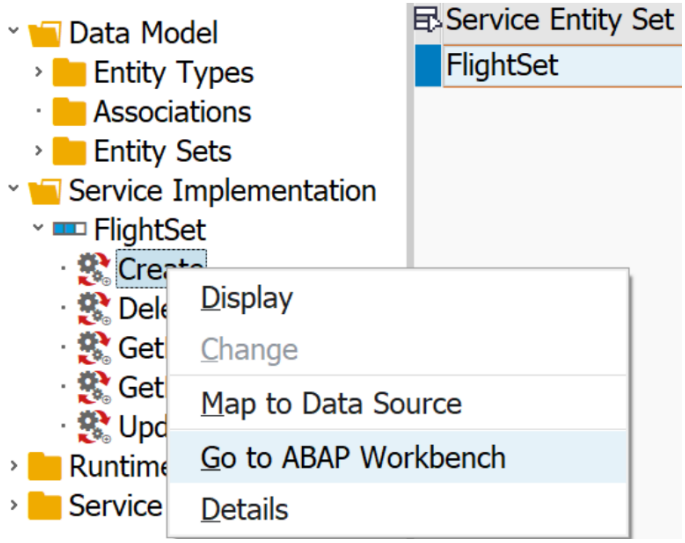
Click **enter**. Choose a package and transport request and wait until your runtime artifacts are automatically created. You can check this in the **Runtime Artifacts** folder, which should now contain different service elements.

Task 2: Implement an OData Service

Short description: Implementing the service functionality in ABAP, which contains the execution of database operations

Generating the service elements in the previous task led to the creation of service templates for the main functionalities to create, read, update and delete (CRUD) data. In order to use these functions, their logic has to be implemented separately in ABAP. In this exercise, we want to create an application which shows all flight data and allows to create a new flight. Therefore, the **Create** and **GetEntitySet** methods, which can be found in the **Service Implementation** folder of your project, have to be implemented.

First, we want to implement the create functionality. Right-click on **Create** within the **FlightSet** in the **Service Implementation** folder and click on **Go to ABAP Workbench**.



Open service implementation

Click **Enter**. Eclipse will open a new tab, in which a code template is contained (your class will have the name ZCL_Z_$$_###_FLIGHT_DPC_EXT, depending on your individual number).

```
class ZCL_Z_000_FLIGHT_DPC_EXT definition
  public
  inheriting from ZCL_Z_000_FLIGHT_DPC
  create public .

public section.
protected section.
private section.
ENDCLASS.



CLASS ZCL_Z_000_FLIGHT_DPC_EXT IMPLEMENTATION.
ENDCLASS.
```

If you already completed the exercise about **ABAP Objects**, you might be familiar with the code structure and syntax. First, there is the class definition, in which methods to be

implemented are defined. Second, there is the class implementation, which contains the implementation of the defined methods.

Since we want to implement the **create** functionality at first, please add the following code line to the **protected section** in the class definition (ignore the error message at first):

```
METHODS flightset_create_entity REDEFINITION.
```

Thereby, the system is notified that you want to implement a new version of the **create_entity** method, which serves to store new data entries. Now, the method can be implemented. For this, you have to define data elements, assign input data to existing data fields, and finally store new data in the database. Please add the following program code to the implementation part of your class:

Implement create_entity service

```
  METHOD flightset_create_entity.

    DATA: ls_request_input_data TYPE zcl_z_000_flight_mpc=>ts_flight,
          ls_flight             TYPE sflight.

*Read request
    io_data_provider->read_entry_data(
    IMPORTING es_data = ls_request_input_data ).

* Assign flight data values to fields
    ls_flight-carrid = ls_request_input_data-carrid.
    ls_flight-connid = ls_request_input_data-connid.
    ls_flight-currency = ls_request_input_data-currency.
    ls_flight-fldate = ls_request_input_data-fldate.
    ls_flight-paymentsum = ls_request_input_data-paymentsum.
    ls_flight-planetype = ls_request_input_data-planetype.
    ls_flight-price = ls_request_input_data-price.
    ls_flight-seatsmax = ls_request_input_data-seatsmax.
    ls_flight-seatsocc = ls_request_input_data-seatsocc.

*Insert data into table SFLIGHT
    INSERT sflight FROM ls_flight.

  ENDMETHOD.
```

Next, the **get_entityset** method, which returns a list of all flight entries, has to be implemented. For this, first the system has to be notified that you want to redefine this method. Please add the following line to the **protected section** of the class definition:

```
METHODS flightset_get_entityset REDEFINITION.
```

Afterwards, add the following program code to the **implementation** of your class:

Implement get_entityset service

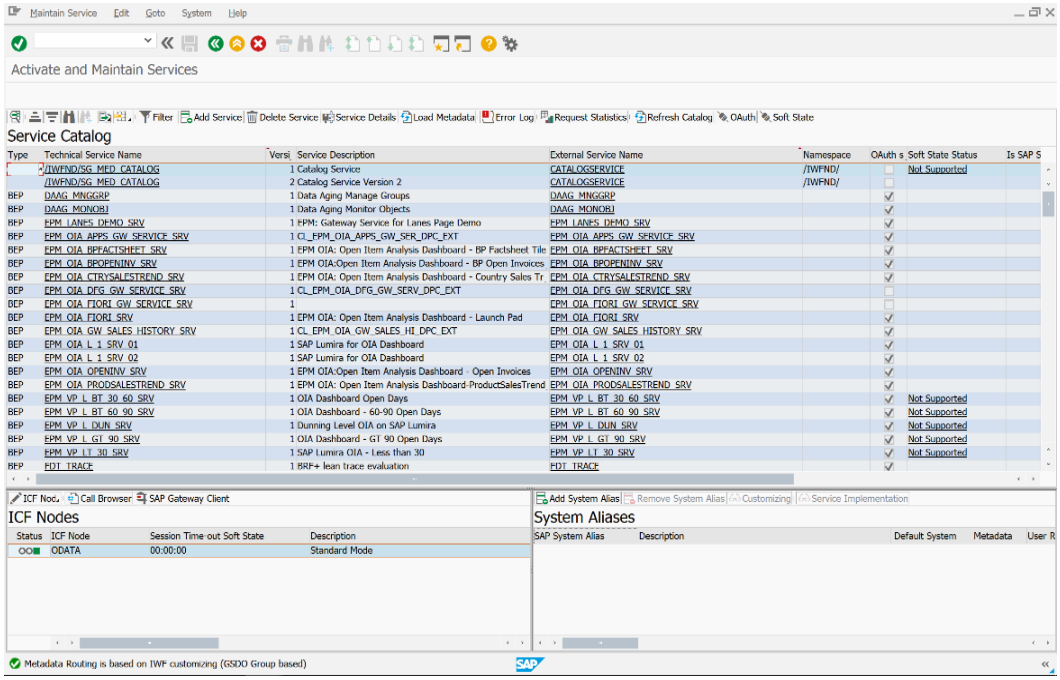
```
METHOD flightset_get_entityset.

DATA it_flight TYPE TABLE OF sflight.
DATA wa_flight TYPE sflight.

SELECT * FROM sflight INTO TABLE it_flight.
DELETE ADJACENT DUPLICATES FROM it_flight COMPARING carrid connid.

et_entityset = it_flight.

ENDMETHOD.
```

Save your program code by clicking on  . Afterwards, activate it by clicking on  in the Eclipse toolbar and **Activate**.

Task 3: Activate and test an OData Service

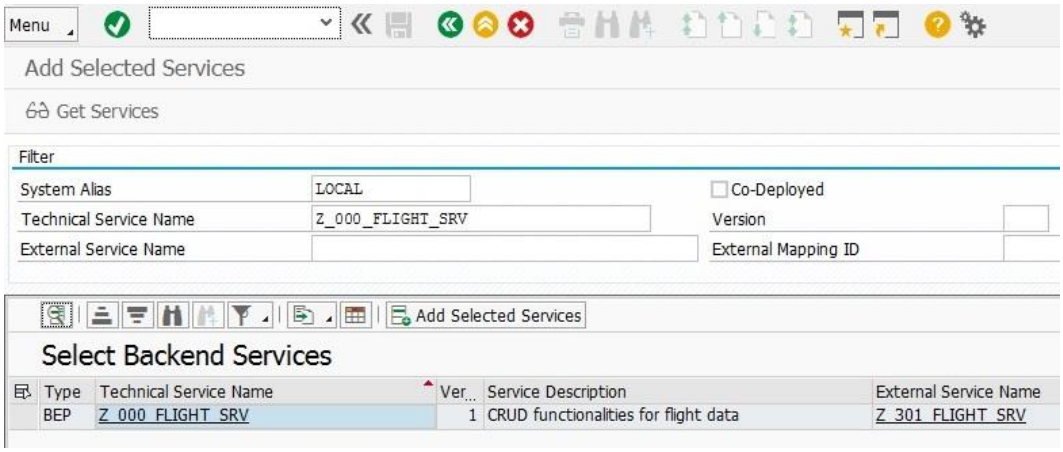
Short description: Activate an OData service and test it by looking at the OData model

To activate services and make them accessible for others, the OData model has to be generated out of the service definition and implementation. For this, please call transaction **/n/IWFND/MAINT_SERVICE** in the integrated SAP GUI. An overview of all activated services appears.



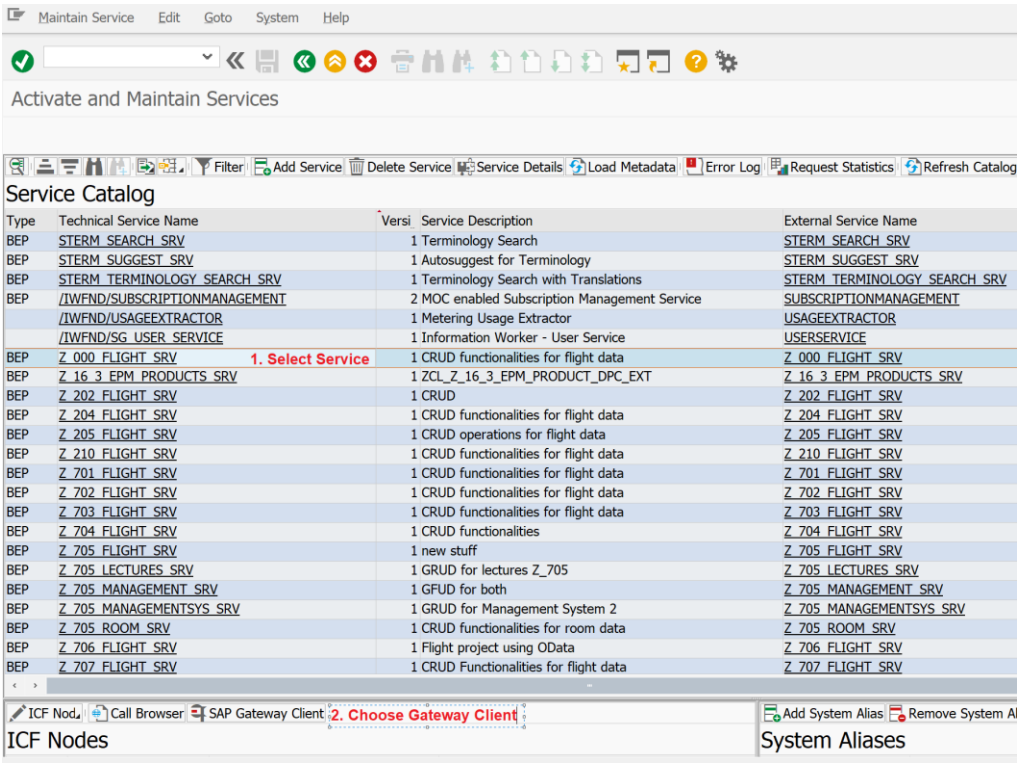
Activate service

In the toolbar above the **Service Catalog**, choose **Add Service**. A new screen appears. Enter **Local** as System Alias and **Z_$$_###_FLIGHT_SRV** as Technical Service Name. Press Enter. Your implemented service should appear in the list.



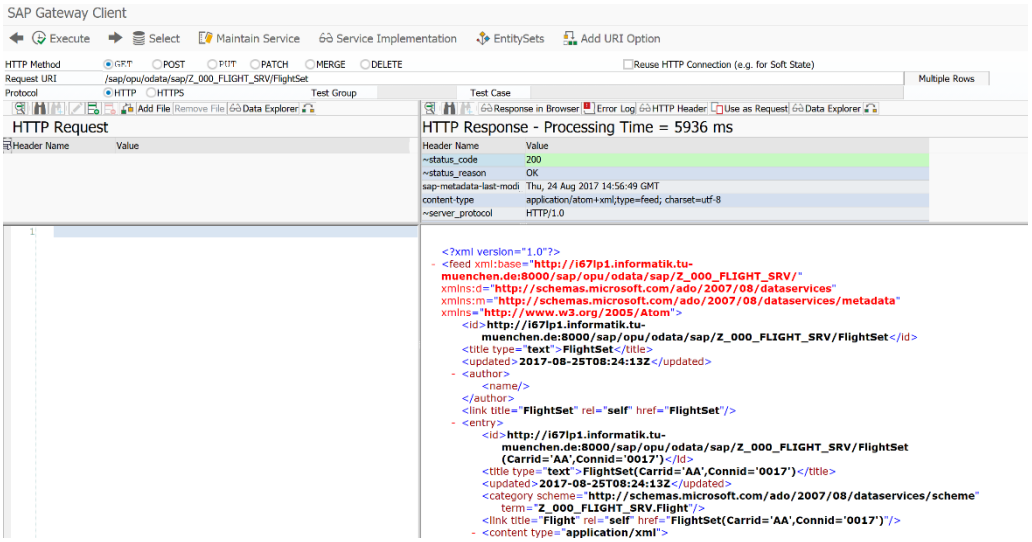
Select your line, and click on **Add Selected Services** on the top of the list. A popup appears which summarizes your selected service. Click **Enter**, and choose a transport request to complete your service activation. You may need to choose your transport request several times, since more than one object has to be activated. Go back to the Service Catalog by clicking on  . Now your service should appear in the list.

Now we want to test the activated service. For this, search for your service **Z_$$_###_FLIGHT_SRV** in the list. Select it by double-clicking on it. Afterwards, click on **SAP Gateway Client** on the bottom of the table. A new SAP GUI window will open.



Test service

With the Gateway client, you can test all services and look at the OData service you created with your implementation. For this, you simply have to insert a URI and execute an HTTP request which calls this URI. In this exercise, we want to test if the implementation of the **Get_EntitySet** method was correct. Therefore, in the application toolbar of the SAP GUI, click on **EntitySets**, and choose **FlightSet**. You may notice that the URI of the Gateway Client changes. Afterwards, click on **Execute** in the Application Toolbar. After a short loading time, you should get a collection of all flight data in the FlightSet in OData format as HTTP response.

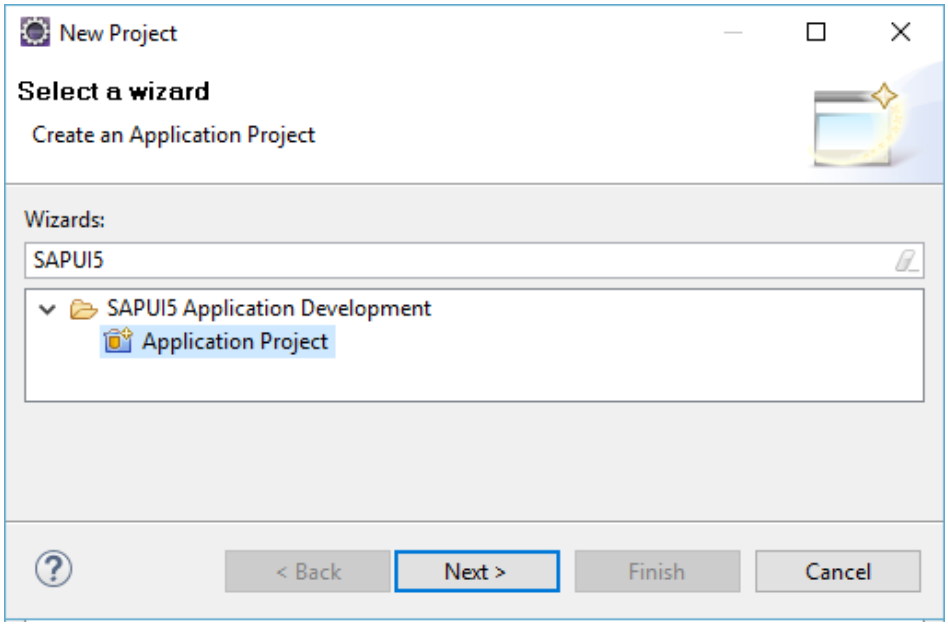


You can use the SAP Gateway client to test HTTP requests with different URI's which call different functions. It is a powerful tool to test your services before you implement e.g. a SAPUI5 application which shall consume your service. Also, it allows you to debug your service calls.

Task 4: Create an SAPUI5 view which consumes an OData service

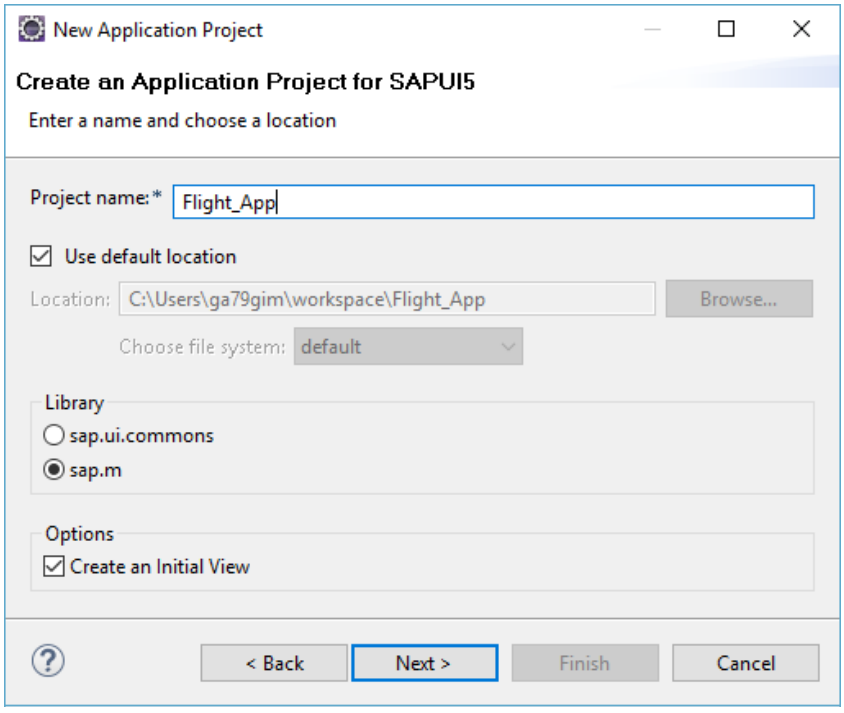
Short description: Creating an SAPUI5 view, which contains a table showing all flight data, and offering the possibility to create new flight data

Back in Eclipse, we now want to create a SAPUI5 application which consumes our OData Services. In the context menu of the Project Explorer, choose **New->Other->SAPUI5 Application Development->Application Project.** Click **Next**.

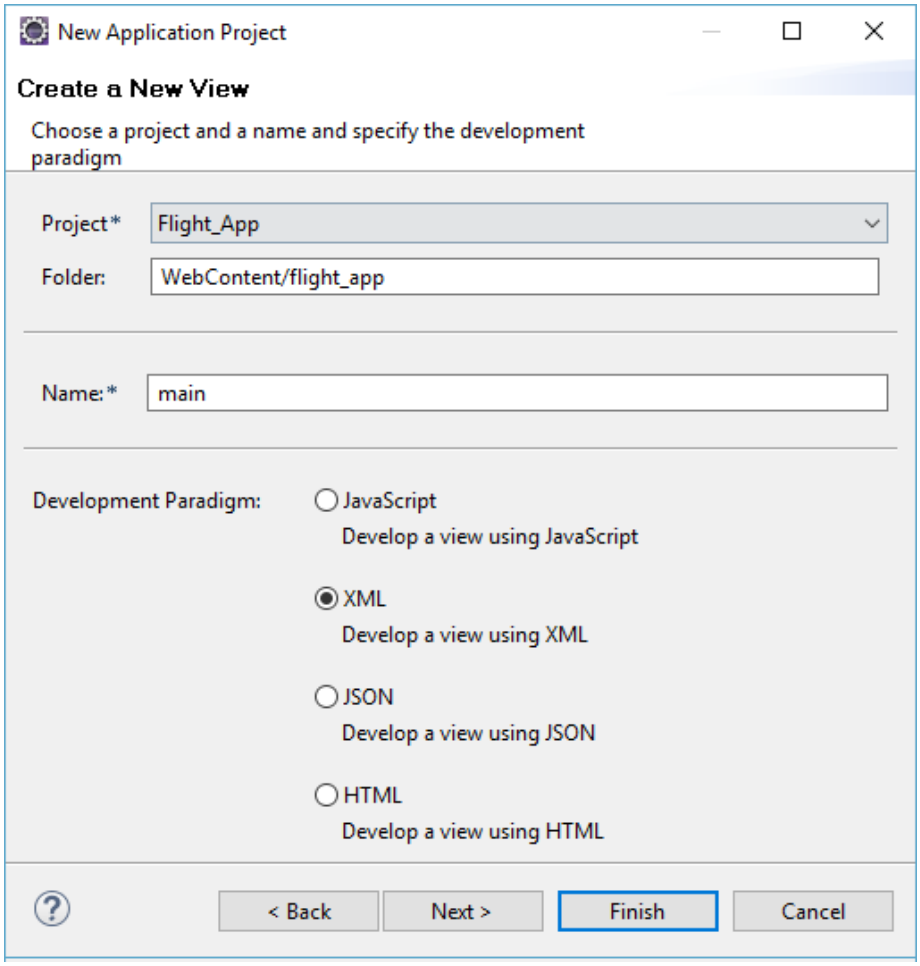


Create SAPUI5 application

Give your project the name **Flight_App** (no need to add your two-digit number at this point). Choose **sap.m** as library and check **Create an Initial View**, and click **Next**.

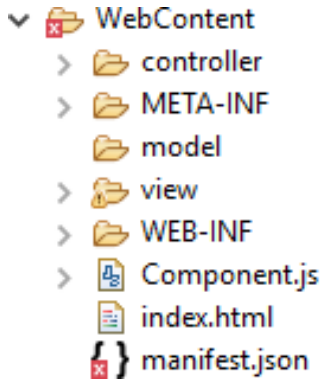


On the next page, you have to define the properties for your initial view. Enter the name **main**, and select to develop the view using **XML**. Developing SAPUI5 views using XML is similar to HTML, but XML is more restrictive and therefore clearer in its syntax. Click on **Finish** to create your project. If you are asked if you want to change your perspective, click **Yes**.



Create a new view

After the project has been created, change your folder structure as depicted on a screenshot below – create the model, view, and controller folders, as well as Component.js and manifest.json files (ignore the error in manifest.json for now). Move the main view and main controller into the corresponding folders. Delete the flight_app folder.



Make the necessary changes to the index.html file (replace 000 with your number).

```
<!DOCTYPE HTML>
<html>
    <head>
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta http-equiv='Content-Type' content='text/html;charset=UTF-8'/>
        <title>Flight OData Service</title>
        <script
                src="resources/sap-ui-core.js"
                id="sap-ui-bootstrap"
                data-sap-ui-theme="sap_bluecrystal"
                data-sap-ui-libs="sap.m"
                data-sap-ui-bindingSyntax="complex"
                data-sap-ui-compatVersion="edge"
                data-sap-ui-preload="async"
                data-sap-ui-resourceroots='{"flight_app_000": "./"}'
                >
        </script>

        <script>

            sap.ui.getCore().attachInit(function () {
                sap.ui.require([
                    "sap/m/Shell",
                    "sap/ui/core/ComponentContainer"
                ], function (Shell, ComponentContainer) {
                    new Shell({
                        app : new ComponentContainer({
                            name : "flight_app_000",
                            settings : {
                                id : "webapp"
                            }
                        })
                    }).placeAt("content");
                });
            });

        </script>

    </head>
    <body class="sapUiBody" role="application" id="content">
    </body>
</html>
```

Define your component in the **Component.js**. For the meaning of single commands refer to the previous exercise. Again, replace 000 with your number.

```
jQuery.sap.declare("flight_app_000.Component");
sap.ui.define([
    "sap/ui/core/UIComponent"
], function (UIComponent) {
    "use strict";

    return UIComponent.extend("flight_app_000.Component", {

        metadata: {
            manifest: "json"
        },

        init: function () {
            // call the init function of the parent
            UIComponent.prototype.init.apply(this, arguments);
            this.getRouter().initialize();
        },

        createContent: function(){
            var oViewData = {
                    component : this
            };

            return sap.ui.view({
                viewName:"flight_app_000.view.App",
                type: sap.ui.core.mvc.ViewType.XML,
                viewData: oViewData
            });
        }
    });

});
```

Implement the application metadata in the manifest.json file. Again, replace 000 with your number.

```
{
    "sap.app": {
        "_version": "1.1.0",
        "id": "flight_app_000",
        "type": "application",
        "applicationVersion": {
            "version": "1.1.0"
        },
        "title": "Flight Monitor"
    },
    "sap.ui5": {
        "rootView": {
            "viewName": "flight_app_000.view.main",
            "type": "XML",
            "async": true
        },
        "models": {
           "Settings": {
                "type": "sap.ui.model.json.JSONModel",
                "uri": "model/applicationProperties.json"
            }
        },
        "routing": {
            "config": {
                "viewPath": "flight_app_000.view",
                "controlAggregation": "pages",
                "routerClass": "sap.m.routing.Router",
                "viewType": "XML",
                "transition": "slide",
                "async": true
            },
            "routes": [{
                "pattern": "",
                "name": "home",
                "target": "home"
            }],
            "targets": {
                "home": {
                    "viewId": "main",
                    "viewName": "main",
                    "controlId": "app",
                    "viewLevel" : 1
                }
            }
        }
    }
}
```

The only missing file to make the project runnable is App view. Create it in the view folder. No controller is needed this time.

```
<mvc:View
    xmlns="sap.m"
    xmlns:mvc="sap.ui.core.mvc"
    displayBlock="true">
    <App id="app"/>
</mvc:View>
```

Now open the main.view.xml. In this view, you have to define which content you want to show in your SAPUI5 application. At first, change the controller name, replacing 000 with your number:

```
controllerName="flight_app_000.controller.main"
```

After that change the title of the application to **Flights**.

```
<Page title="Flights">
```

Now we want to define a table, which shows all current flight data in one central table. For this, we have to define column headers and rows, which are linked to the OData service we created in the previous step. In order to define the table with its connection to the OData service, add the following code between the <content> tags of your xml view:

```
<Table id="flightsTable" items="{/FlightSet}" growing="true"
    growingScrollToLoad="true">
</Table>
```

Next, we want to define the column header of the table. This has to be done manually with column tags. Therefore, add the following code between the <Table></Table> tags:

```
<columns>
    <Column>
        <Text text="Airline ID" />
    </Column>
    <Column>
        <Text text="Connection ID" /
    </Column>
    <Column>
        <Text text="Date" />
    </Column>
    <Column>
        <Text text="Price" />
    </Column>
    <Column>
        <Text text="Currency" />
    </Column>
    <Column>
        <Text text="Plane Type" />
    </Column>
    <Column>
        <Text text="Max. Seats" />
    </Column>
    <Column>
        <Text text="Booked Seats" />
    </Column>
    <Column>
        <Text text="Payment Sum" />
    </Column>
</columns>
```

Define view elements

Afterwards, we can decide the content we want to show from our defined OData services. For this, links to the attribute names have to be defined. Add the following code after the columns definitions:

```
<items>
    <ColumnListItem>
        <cells>
            <Text text="{Carrid}" />
            <Text text="{Connid}" />
            <Text text="{Fldate}" />
            <Text text="{Price}" />
            <Text text="{Currency}" />
            <Text text="{Planetype}" />
            <Text text="{Seatsmax}" />
            <Text text="{Seatsocc}" />
            <Text text="{Paymentsum}" />
        </cells>
    </ColumnListItem>
</items>
```

Finally, we want to add a footer which shows a button with the functionality to create a popup. This popup will contain dynamically created fields which serve to create new flight data. Therefore, after the </content>-tag, add the following code:

```
<footer>
    <Bar>
        <contentLeft>
            <Button text="Create new Flight" type="Accept" press="createPopup"/>
        </contentLeft>
    </Bar>
</footer>
```

Define footer

Save your view by clicking on one of the Save buttons  .

Task 5: Create an SAPUI5 controller which consumes an OData service

Short description: Creating an SAPUI5 controller, which fills the table with flight data, dynamically creates a popup, and offers the functionality to create new flight data.

You might ask why we need a model folder when we are using the OData service in this exercise. It will contain the file with the application properties, such as a link to OData service. Thus, create an applicationProperties.json file in the model folder. Add you service URL as oDataUrl parameter:

```
{
    "oDataUrl" : "/sap/opu/odata/sap/Z_000_FLIGHT_SRV/"
}
```

Now open the **main.controller.js** file. Define it as an extension of sap.ui.core.mvc.Controller (replace 000 with your number):

```
sap.ui.define([
    "sap/ui/core/mvc/Controller"
], function (Controller) {
    "use strict";

    return Controller.extend("flight_app_000.controller.main", {

        getRouter : function () {
            return sap.ui.core.UIComponent.getRouterFor(this);
        },


        onInit: function() {

        }
    });

});
```

Read the URL of your OData service from the settings model synchronously and bind the OData model to the view (in the onInit function):

```
onInit: function() {
    var sPath = $.sap.getModulePath("flight_app_000", "/model/applicationProperties.json");
    var that = this;

    var oSettingsModel = new sap.ui.model.json.JSONModel();
    oSettingsModel.loadData(sPath);
    oSettingsModel.attachRequestCompleted(function(){
        that.getView().setModel(this, "Settings");
        var serviceURL = that.getView().getModel("Settings").getProperty("/oDataUrl");
        var oModel = new sap.ui.model.odata.v2.ODataModel(serviceURL);
        that.getView().setModel(oModel);

    });
}
```

After that we want to add a JavaScript function which creates the popup to create new flight data. You may have probably noticed already that at the end of the previous task, we added **createPopup** as event when the button in the footer is pressed. Therefore, add the following code frame to your controller right after the previously created **onInit** function:

```
createPopup : function(){
    //Insert functionality here
},
```

In order to create a dynamically displayed popup, several lines of code have to be created. First, we want to create two buttons which are shown at the bottom of the popup. The first one serves to cancel the creation of new flight data. Please add the following code to the **createPopup** function:

```
var cancelButton = new sap.m.Button({
    text : "Cancel",
    type : sap.m.ButtonType.Reject,
    press : function() {
        sap.ui.getCore().byId("Popup").destroy();
    }
});
```

Create Popup buttons

The second button serves to save newly created flight data. For this, a more extensive code is necessary, which reads user input, and calls the OData service to create a new flight. Hence, please add the following JavaScript code to the **createPopup** function:

```
var saveButton = new sap.m.Button({
    text : "Save",
    type : sap.m.ButtonType.Accept,
    press : function() {
        var serviceURL = that.getView().getModel("Settings").getProperty("/oDataUrl");
        var oModel = new sap.ui.model.odata.v2.ODataModel(serviceURL);
        var oNewFlight = {
            Carrid : sap.ui.getCore().byId("carrid").getValue(),
            Connid : sap.ui.getCore().byId("connid").getValue(),
            Fldate : sap.ui.getCore().byId("fldate").getDateValue(),
            Price : sap.ui.getCore().byId("price").getValue(),
            Currency : sap.ui.getCore().byId("currency").getValue(),
            Planetype : sap.ui.getCore().byId("planetype").getValue(),
            Seatsmax : parseInt(sap.ui.getCore().byId("seatsmax")
                    .getValue()),
            Seatsocc : parseInt(sap.ui.getCore().byId("seatsocc")
                    .getValue()),
            Paymentsum : sap.ui.getCore().byId("paymentsum").getValue()
        };

        oModel.create('/FlightSet', oNewFlight, {
            success : function(oData, oResponse) {
                sap.m.MessageToast.show("Flight successfully created!");

                sap.ui.getCore().byId("Popup").destroy();
            },
            error : function(oError) {
                sap.m.MessageToast.show("Error during flight creation")
            }
        });
    }
});
```

Now, both buttons with the necessary functionalities to react to user input have been created. Please save your JavaScript file at this point.

Finally, the view elements of the popup have to be generated dynamically, containing the input fields and the previously created buttons. For this, each field has to be added manually via JavaScript code. First, add the following code to the **createPopup** function right below your button definitions:

```
var oDialog = new sap.m.Dialog("Popup", {
    title : "Create Flight",
    modal : true,
    contentWidth : "1em",
    buttons : [ saveButton, cancelButton ],
    content : [  ]
});
sap.ui.getCore().byId("Popup").open();
```

Define Popup fields

In the **content** section (between "**[**" and "**]**") of your dialog, the single input fields and labels have to be defined. Therefore, please add the following code lines to this section (starting from the left column, adding the content from the right column below):

```
new sap.m.Label({
    text : "Airline ID"
}), new sap.m.Input({
    maxLength : 20,
    id : "carrid",
    placeholder : "e.g. LH"
}), new sap.m.Label({
    text : "Connection ID"
}), new sap.m.Input({
    maxLength : 20,
    id : "connid",
    placeholder : "e.g. 11"
}), new sap.m.Label({
    text : "Date"
}), new sap.m.DateTimeInput({
    id : "fldate",
    placeholder : "e.g. Jul 15, 2016"
}), new sap.m.Label({
    text : "Price"
}), new sap.m.Input({
    maxLength : 20,
    id : "price",
    placeholder : "e.g. 111.11"
}), new sap.m.Label({
    text : "Currency"
}), new sap.m.Input({
    maxLength : 20,
    id : "currency",
    placeholder : "USD"
```

```
}), new sap.m.Label({
    text : "Plane Type"
}), new sap.m.Input({
    maxLength : 20,
    id : "planetype",
    placeholder : "e.g. 747-300"
}), new sap.m.Label({
    text : "Max. Seats"
}), new sap.m.Input({
    maxLength : 20,
    id : "seatsmax",
    placeholder : "e.g. 300"
}), new sap.m.Label({
    text : "Booked Seats"
}), new sap.m.Input({
    maxLength : 20,
    id : "seatsocc",
    placeholder : "e.g. 250"
}), new sap.m.Label({
    text : "Payment Sum"
}), new sap.m.Input({
    maxLength : 20,
    id : "paymentsum",
    placeholder : "e.g. 1111.11"
}),
```

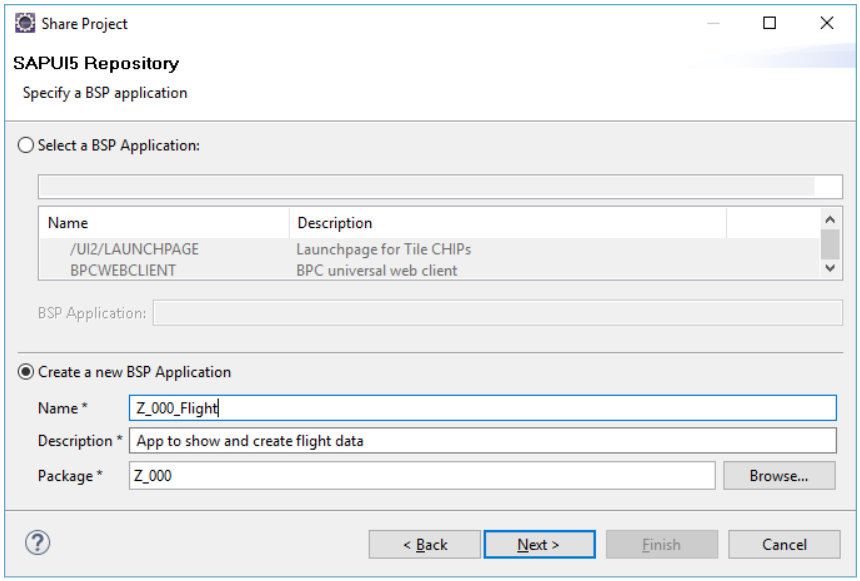
Finally, save your JavaScript file by clicking on  .

Task 6: Deploy your application to the SAP backend

Short description: Deploying the application to the SAP backend to make it accessible on the server

To deploy your application to the SAP backend, right-click on your web project in the project explorer and choose **Team->Share Project.** In the next step, select **SAPUI5 ABAP Repository** as repository type. Afterwards, choose your system connection, and enter User and Password if necessary. Next, define to create a new BSP Application. Enter Name **Z_$$_###_Flight**, a description, and your Package to which you want to assign your application. Click **Next**.

Deploy application to SAP backend

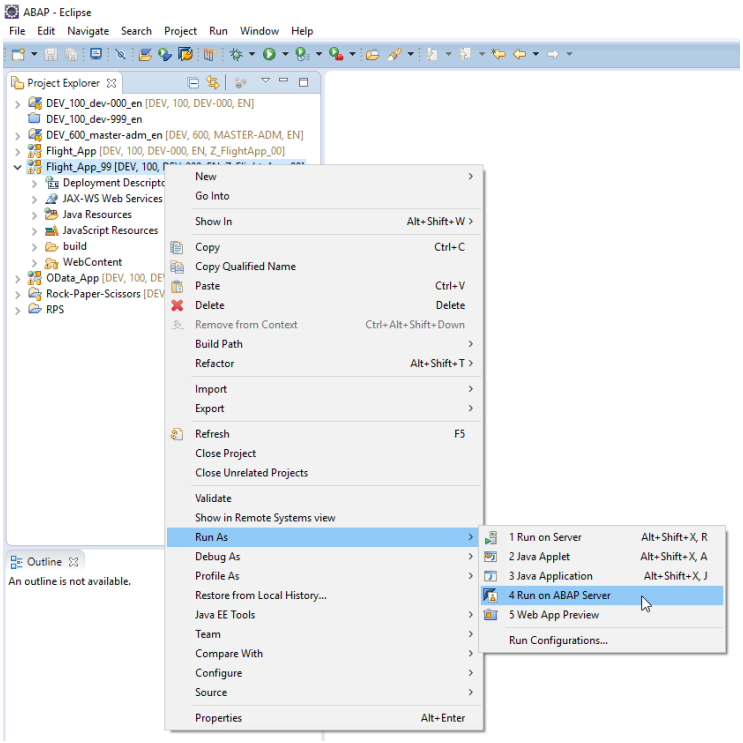


Choose a transport request. Again, you may use your default transport request you used for other exercises. Click on **Finish**. The application will be deployed to the SAP backend.

Submit repository objects

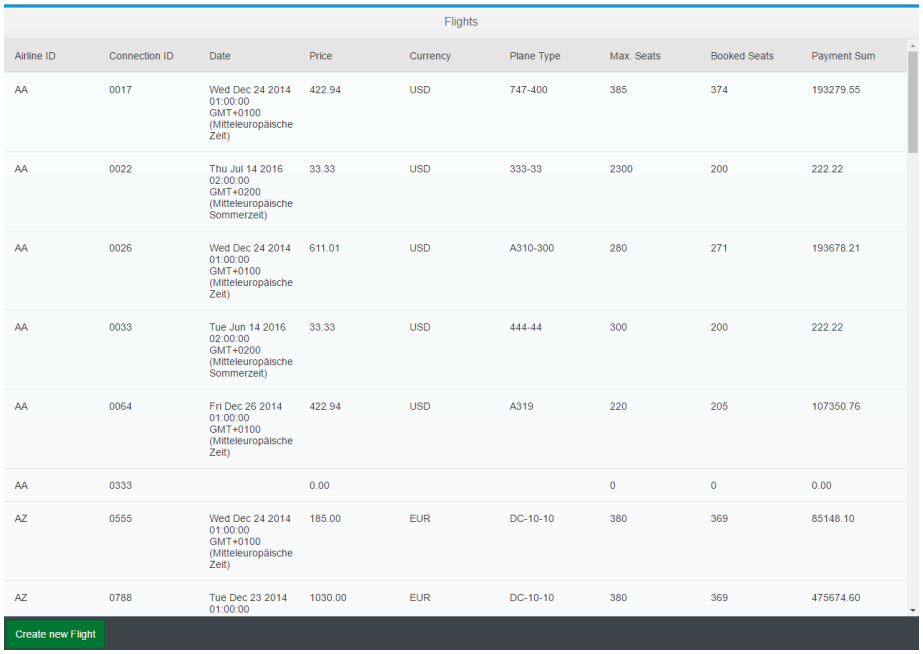
To test your application, first make sure that all source files were correctly submitted. For this, right-click on your project, and click on **Team->Submit**. Choose all your files and click on **Finish**. If **submit** is already greyed out, everything is fine.

Finally, right-click on your project, and choose **Run As->Run on ABAP Server**.

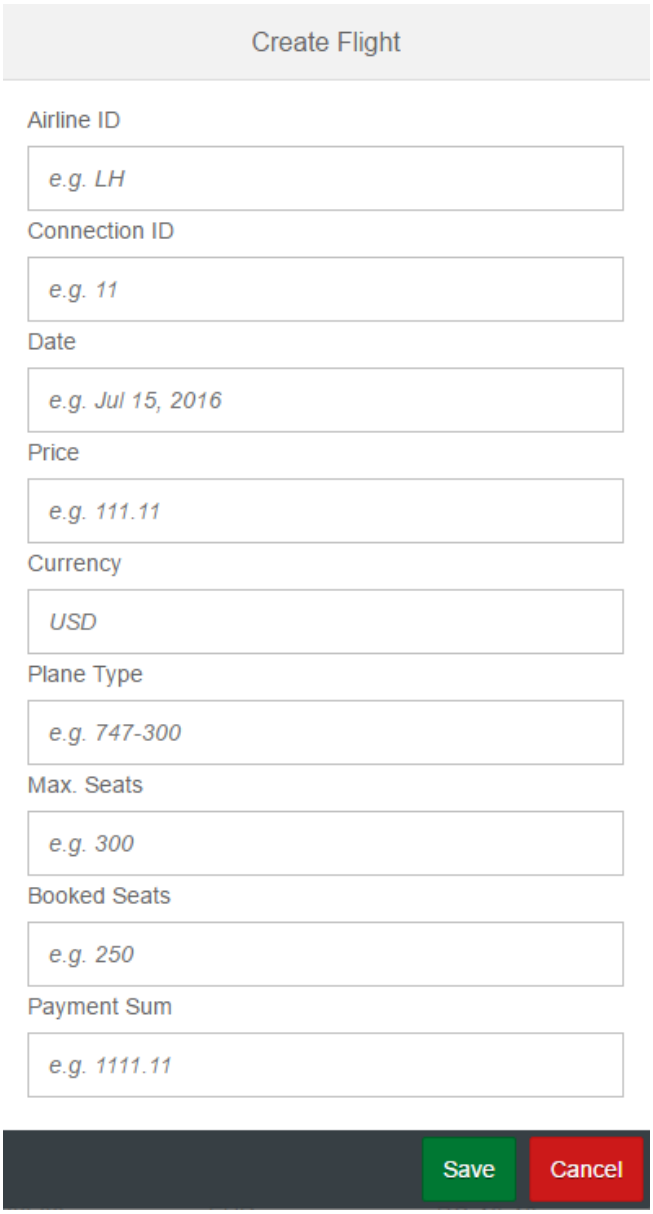


Test your application

You may need to login with your username and password. Afterwards, your application will be displayed in the integrated web browser. There, you get an overview of all flight data in the system.



On the bottom left, there is a button to create a new flight. When creating new flight data, please pay attention that we did not implement any error handling. Therefore, please test the flight creation with data similar to the placeholder values. Otherwise, you will get an error message. If you implemented the service and entered the values correctly, after clicking on save, your newly created flight will be shown in the flight list.



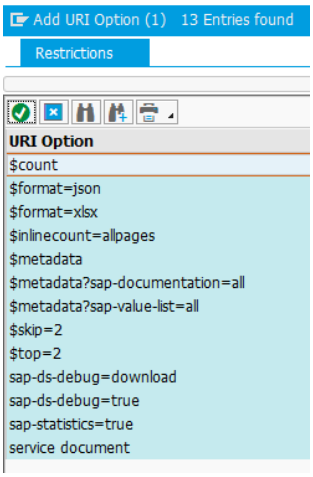
Task 7: Make use of query options

Short description: Get to know the built in OData query options to adapt the results of an OData query to your specific needs.

We are going to use query parameters to shape the data returned by the OData service. Of course, this could also be done within the controller, but this approach has a better performance in terms of runtime and dataload.

To get an impression of query options, call transaction **/n/IWFND/MAINT_SERVICE** and open your OData service in the SAP Gateway Client, which has already been introduced to you in task 3. If you are more comfortable in your browser, feel free to choose Call Browser. In that case you will be taken to the URL **<host>:<port>/sap/opu/odata/sap/Z_$$_###_FLIGHTEXT_SRV/** with ### being your individual number and $$ the institution id.

Select FlightSet from EntitySets and execute Execute this HTTP-Request to display the entries of FlightSet in XML. You can display the entries in json with **/FlightSet/?$format=json.** Click the button **"add URI Option"** Add URI Option to show which options are currently supported by OData.



The options can be added to the request URI by double-clicking on the respective entry. Alternatively you can extend the URI manually following the pattern:

```
/FlightSet?$option
```

Multiple options can be concatenated using **&**, e.g.

```
/FlightSet?$option1&$option2.
```

Only **$count** is treated differently, because it is considered as a resource, not as a query option (**/FlightSet/$count).**

The query options **$count** and a **$select** are always usable as soon as you have created and activated an ODataService (e.g. /FlightSet?$select=Price,Currency). Further options like **$filter** and **$expand** have to be implemented in the dpc first. Implementation and usage of the query options is showcased in this exercise.

First, in the **main.view.xml** enhance the **Bar** in the **footer** of the page with the following code:

```
<contentMiddle>
    <Text text="Average Price:" id="avgPrice" />
</contentMiddle>
```

This text element will contain the calculated average price of the flights. In order to calculate the price place the following function in your **main.controller**. Be sure to use the URL of your service in the function **"/sap/opu/odata/sap/Z_$$_###_FLIGHT_SRV**":

```
determineAvgPrice : function() {
    var serviceURL = this.getView().getModel("Settings").getProperty("/oDataUrl");
    var oModel = new sap.ui.model.odata.v2.ODataModel(serviceURL);
    var that = this;
    oModel.read("/FlightSet", {
        urlParameters : {
            "$select" : "Price"
        },
        success : function(oData, oResponse) {
            console.log(oData);
            var res = oData.results;
            var sum = res.reduce(function(pv, cv) {
                return pv + parseFloat(cv.Price);
            }, 0.0);
            var average = sum / res.length;
            average = average.toFixed(2);
            var oLabel = that.getView().byId("avgPrice");
            oLabel.setText("Average Price: " + average);
        }
    });
},
```

The function first reads the whole EntitySet of our Flights Table in our UI5 application. However, since we are only interested in the price we do not read all columns/fields but use our select option instead. The data load to transmit will be hence much lesser. After the data items were loaded, the success callback function is triggered which sums up the price information and divides the result by the number of entries to calculate the average price. The average price is finally rounded to 2 decimal places and displayed on the label.

Hint: Of course it would be even more efficient to calculate the average price on the database and transmit only this single number. We even could easily do this via the so-called function import which you will see later in this exercise. However, this example shall just show you how you can work with the select option and how you can process data in the front end layer.

In order to calculate the function each time the view is generated, we have to trigger the function in our **onInit** function. Add the following line to the end of the onInit function in your **main.controller**.

```
this.determineAvgPrice();
```

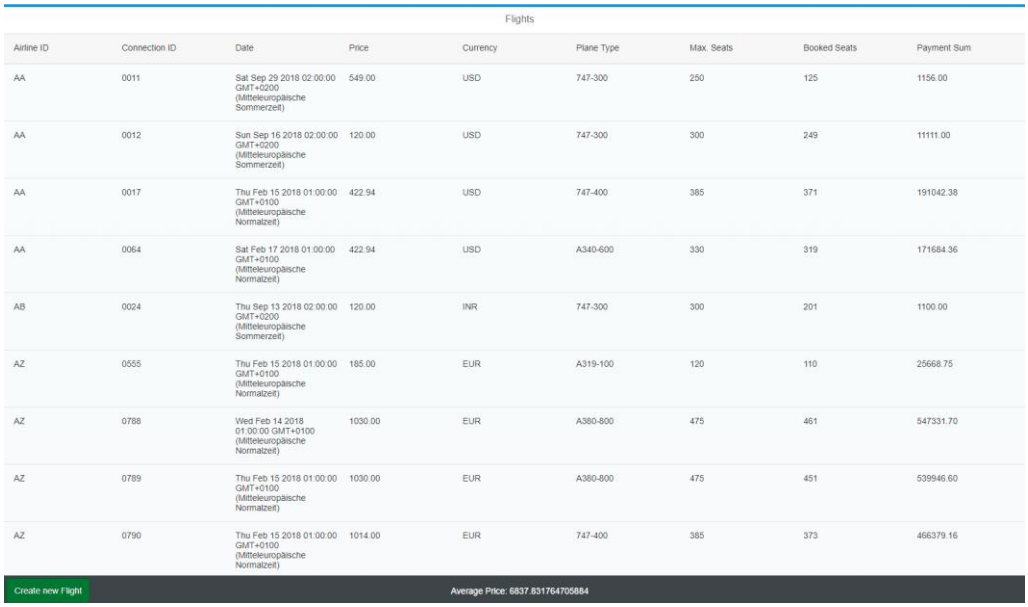
The calculation also has to be redone when a new flight is created. For that purpose, define the **that** variable at the beginning of the **createPopup** function

```
var that = this;
```

and call the new function in a success-callback of the flight creation. It has to look as below:

```
oModel.create('/FlightSet', oNewFlight, {
    success : function(oData, oResponse) {
        sap.m.MessageToast.show("Flight successfully created!");

        that.determineAvgPrice();

        sap.ui.getCore().byId("Popup").destroy();
    },
    error : function(oError) {
        sap.m.MessageToast.show("Error during flight creation")
    }
});
```

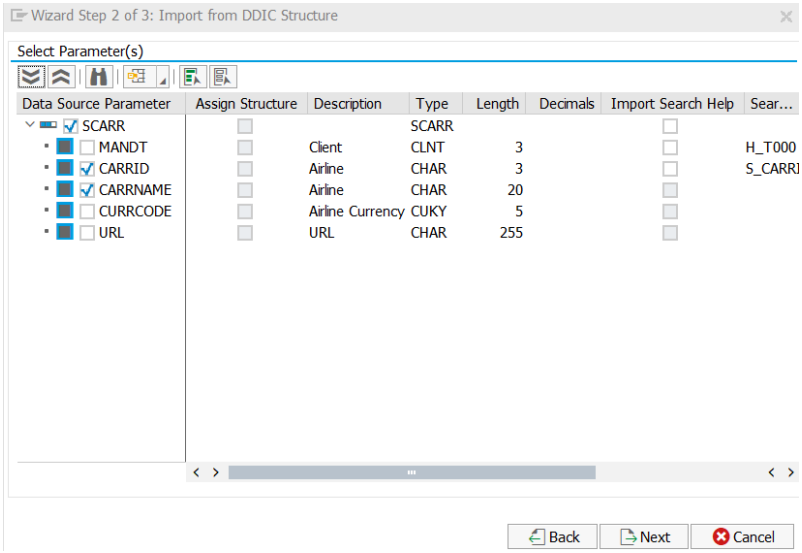
Now you will see the text label with calculated average price:



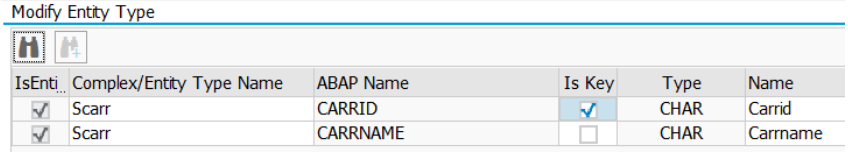
Task 8: Make use of expansion

Short description: As you may now from our previous chapter "Database Access", the SFLIGHT table is connected with the table SCARR via a foreign key relationship. In this task you will learn about the navigation and association capabilities of OData in order to expand the information provided by SFLIGT with data that is stored SCARR. This is similar to a table join on the database layer.

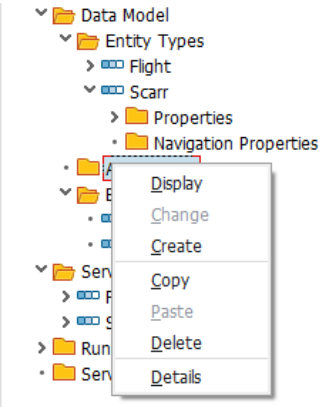
In the integrated SAP GUI, call transaction code **SEGW** to open the gateway service builder. In your project that you should see there import a new DDIC structure, as described in Task 1. Its name should be **Scarr**, and it should be based on the ABAP Dictionary Structure **SCARR**. For the fields, select **Carrid** and **Carrname** and click on **Next.**



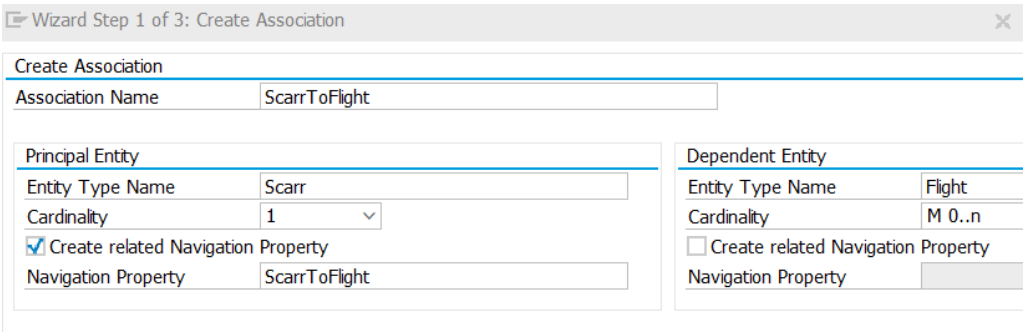
Select **Carrid** as your key field and click on **Finish**.



After that you have to create an association between both the two entities and the entity sets. An association is a connection link the specifies the cardinality between the respective entities similar to a foreign-key relationship. Right-click on **Associations** and then on **Create**.

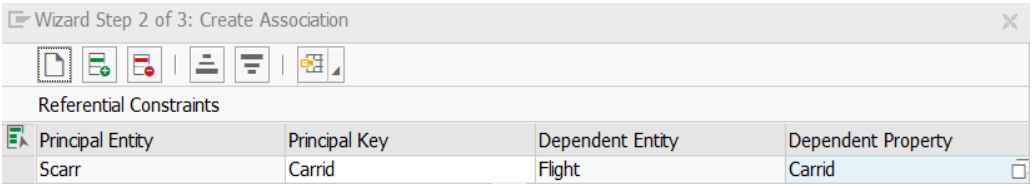


Name the association **ScarrToFlight**, choose **Scarr** as a principal entity with a cardinality **1** and **Flight** as a dependent entity with cardinality **0..n**. Check the **'Create a navigation property'** only for the principal entity and give it the same name as for the association itself ('**ScarrToFlight**'). Afterwards, click on **Next**.



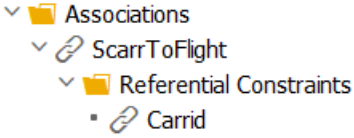
The navigation describes the direction via which you can expand the information. In our case we will read information from Scarr (Principal) and extend this information with data from Flight (Dependant Entity). Of course you could also create a second Navigation property to move from Flight to Scarr.

The next screen asks you to which field/property of the Flight entity the principal key is referring to. Choose **Carrid** both, the principal key and the dependent property. Afterwards, click on **Next**.

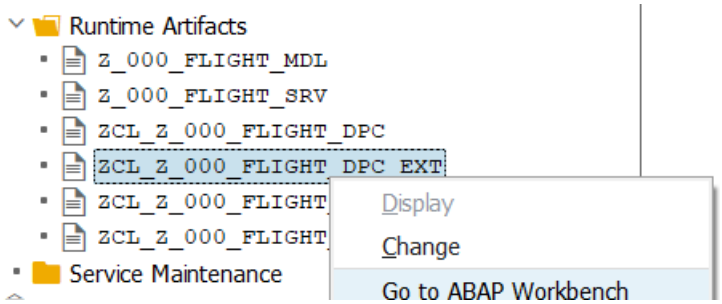


Click on Finish in the final window. Save your project and generate runtime objects by clicking on **Generate**.

Both entities are now connected via the association



However, you probably remember that in order to read our Flightset information we first had to implement the **Get_FlightEntitySet** Method. The **$expand** query option the we will later use to do the OData version of a table join requires that all getters (Get_Enitity and Get_EntitySet) for the respective entities are already implemented. Therefore, our next step is to implement these methods. Right-click on …DPC_EXT class in the Runtime Artifacts section and choose **Go to ABAP Workbench**.



Since we already implemented the **flightset_get_entityset** method in **Task3** we only have to implement the **get_entityset** method for Scarr as well as the **get_entity** methods for Flight & Scarr. To do so, please add the following code lines in the class definition in the protected section.

```
METHODS flightset_get_entity REDEFINITION.
METHODS scarrset_get_entity REDEFINITION.
METHODS scarrset_get_entityset REDEFINITION.
```

We are now going to implement the get_entity methods for Scarr and Flight. In contrast to the **get_entityset** method, the **get_entity** only returns a single entry of the database table. Therefore the user has to specify the key fields of the table entry in his request URI in the form:

```
/FLIGHTSet(Carrid='AA',Connid='0011')
```

Our implementation will hence first read the sent parameters from the GET request and perform the SQL select according to this information. As you can see from the above example, the parameters are provided as **key-value pairs**. The parameter that contains this information is **it_key_tab**. We will first read a single entry of this table to get a flat structure variable. After that we will read the value field of the structure to get the actual key information. The result of our SQL query is returned in the parameter **er_entity**. If you are interested in which other parameters are already defined by the parent class **_DPC**, inspect ZCL_Z_$$_###_FLIGHT_DPC in your Runtime Artifacts.

Add the following code into the **IMPLEMENTATION** part of your **_DPC_EXT** class.

```
METHOD flightset_get_entity.
  DATA: lv_carrid TYPE sflight-carrid.
  DATA: lv_connid TYPE sflight-connid.
  IF it_key_tab IS NOT INITIAL.
    TRY.
        DATA(ls_key_carrid) = it_key_tab[ name = 'Carrid' ].
        lv_carrid = ls_key_carrid-value.
        DATA(ls_key_connid) = it_key_tab[ name = 'Connid' ].
        lv_connid = ls_key_connid-value.
        SELECT SINGLE *
          FROM sflight
          INTO CORRESPONDING FIELDS OF er_entity
         WHERE carrid = lv_carrid AND connid = lv_connid.
      CATCH cx_root.
        RETURN.
    ENDTRY.
  ENDIF.
ENDMETHOD.
```

Hint: The **INTO CORRESPONDING FIELDS OF** statement is a nice way to let your runtime environment decide which fields of the source structure have to be copied into your target structure. Hence you do not have to make the assignment manually for each field.

Continue with the implementation of the **scarrset_get_entity**. This Method works much the same way as the flightset_get_entity. However, since our Scarr entity only has one Key property we only have to read one field from the **it_key_tab** table.

```
METHOD scarrset_get_entity.
  DATA: lv_carrid TYPE scarr-carrid.
  IF it_key_tab IS NOT INITIAL.
    TRY.
        DATA(ls_key_carrid) = it_key_tab[ name = 'Carrid' ].
        lv_carrid = ls_key_carrid-value.
        SELECT SINGLE *
          FROM scarr
          INTO CORRESPONDING FIELDS OF er_entity
         WHERE carrid = lv_carrid.
      CATCH cx_root.
        RETURN.
    ENDTRY.
  ENDIF.
ENDMETHOD.
```

Finally, implement the get_entityset method for Scarr.

```
  METHOD scarrset_get_entityset.
    IF it_key_tab IS NOT INITIAL.
    ELSE.
      SELECT *
      FROM scarr
      INTO CORRESPONDING FIELDS OF TABLE et_entityset.
    ENDIF.
  ENDMETHOD.
```

Until now we have used the request `/FLIGHTSet` to read the whole entityset into our flight monitor application (**Task5**). This request does not require and key information since we dont want to specify any certain data item as we do in our get_entity methods.

However, our new navigation property we implemented at the beginning of this task requires to pass **Carrid** information to our **dependent entity Flight**. This is because we would like to expand the information we get from Scarr with information from Flight. The "**join**" information we have to use for this expansion is the field **Carrid** (our foreign-key). However, the flightset_get_entityset method can't handle such key parameters. It is hence necessary to modify the existing query-method to allow the selection and filtering of entities. Change your implementation of the **flightset_get_entityset** method according to the following snippet:

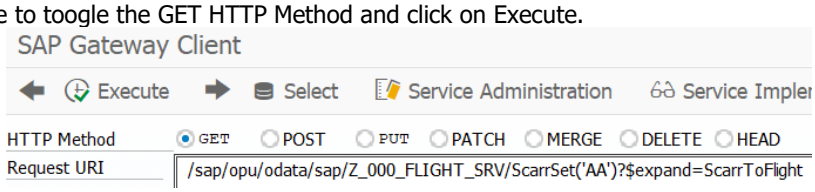
```
IF it_key_tab IS NOT INITIAL.
  DATA: lv_carrid TYPE sflight-carrid.
  TRY.
      DATA(ls_key_carrid) = it_key_tab[ name = 'Carrid' ].
      lv_carrid = ls_key_carrid-value.
      SELECT *
        FROM sflight
        INTO CORRESPONDING FIELDS OF TABLE et_entityset
       WHERE carrid = lv_carrid.
      DELETE ADJACENT DUPLICATES FROM et_entityset COMPARING carrid connid.
    CATCH cx_root.
      RETURN.
  ENDTRY.
ELSE.
  SELECT *
    FROM sflight
    INTO CORRESPONDING FIELDS OF TABLE et_entityset.
  DELETE ADJACENT DUPLICATES FROM et_entityset COMPARING carrid connid.
ENDIF.
```

Save and **activate** your changes.

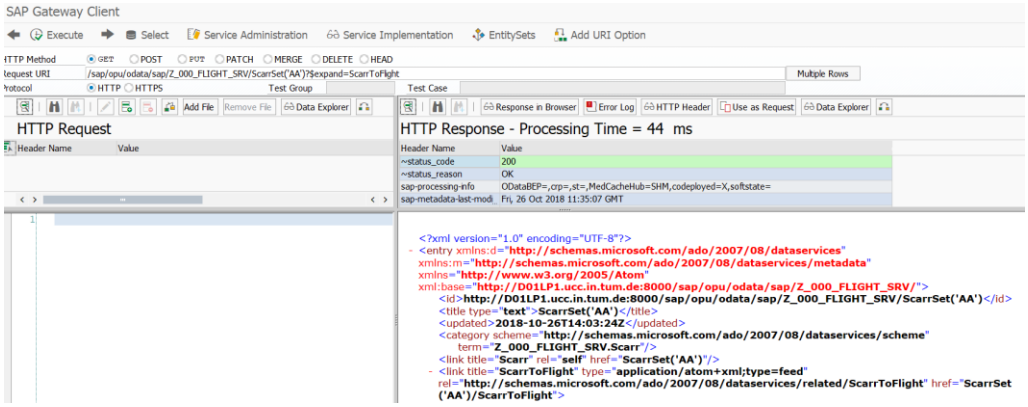
You can test now your implementation using the SAP Gateway Client as described in task 3. Delete the **?$format=xml** part from the URI and add the following option: **ScarrSet('AA')?$expand=ScarrToFlight**. Your complete request URI should look like this:

```
/sap/opu/odata/sap/Z_$$_###_FLIGHT_SRV/ScarrSet('AA')?$expand=ScarrToFlight
```

Make sure to toogle the GET HTTP Method and click on Execute.



You should see now not only the information that the name of **AA** is **American Airlines** but also all flights that are associated with this carrier.



Now you are going to use this function call in our UI5 application. Currently you can only see an overview of all flights with all airlines. But what if you're interested solely in a specific airline? First it is necessary to extend the existing view. Navigate to back to your **Z_$$_###_FLIGHT_APP** via your **project explorer** and open the **main.view.xml** file. Add the following code above the table definition to create a dropdown field with the airlines as well as two buttons. The first button sets the filter, and the second one removes it:

```
<form:SimpleForm layout="ResponsiveGridLayout"
    editable="true" emptySpanXL="0" emptySpanL="0" emptySpanM="0"
    emptySpanS="0" columnsXL="3" columnsL="3" columnsM="3" columnsS="3">
    <Label text="Choose an airline to filter" for="airlines" />
    <ComboBox id="airlines" showSecondaryValues="true" items="{/ScarrSet}">
        <core:ListItem key="{Carrid}" text="{Carrname}"
            additionalText="{Carrid}" />
    </ComboBox>
    <Button type="Emphasized" text="Apply Filter" press="onFilter" />
    <Button type="Reject" text="Clear Filter" press="onClear" />
</form:SimpleForm>
```

You will notice that Eclipse notifies you about some error in your code. This is because you have referenced a new XML namespace **form** without declaring it. Add the line

```
xmlns:form="sap.ui.layout.form"
```

to the namespace declaration at the beginning of the view. Our "filter-button" defines a new event called "onFilter" we are now going to implement. Therefore, switch to the controller and add the following code to it (**be sure to use your URL**):

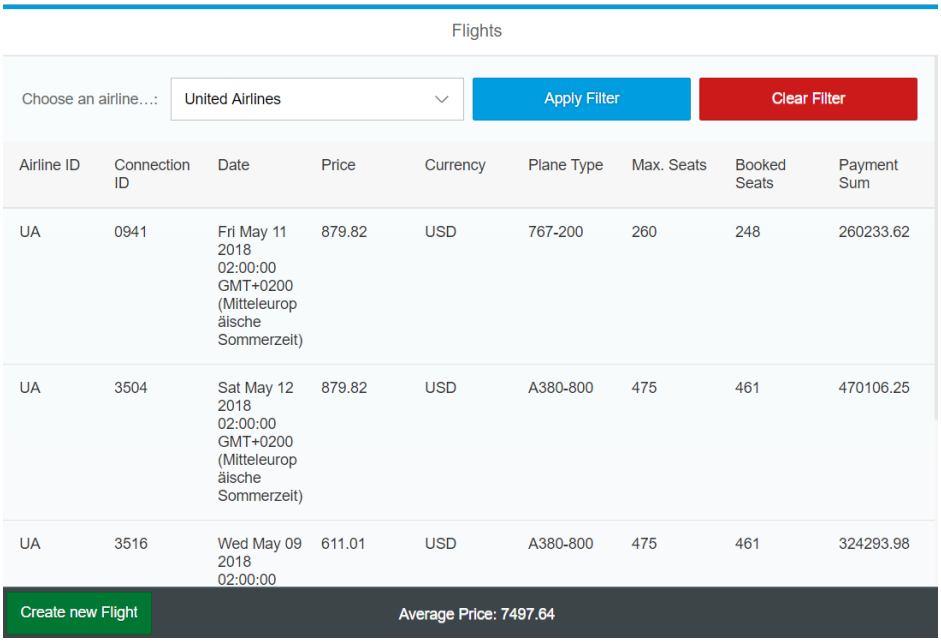
```
onFilter : function() {
    var serviceURL = this.getView().getModel("Settings").getProperty("/oDataUrl");
    var oModel = new sap.ui.model.odata.v2.ODataModel(serviceURL);
    var that = this;
    var id = this.getView().byId("airlines").getSelectedKey();
    if(id !== ''){
        var sPath = "/ScarrSet('"+id+"')";
        oModel.read(sPath, {
            urlParameters : {
                "$expand" : "ScarrToFlight"
            },
            success : function(oData, oResponse) {
                that.getView().byId("flightsTable").removeAllItems();
                var oModelJs = new sap.ui.model.json.JSONModel();
                oModelJs.setProperty("/FlightSet", oData.ScarrToFlight.results);
                that.getView().byId("flightsTable").setModel(oModelJs);
                console.log(oData.ScarrToFlight.results);
            }
        });
    }

},
```

The function first reads the key of the selected airline form the dropdown field "**airlines**". Based on this key information our new OData service is called to return only the flights for the selected carrier. On success (callback function) our existing table content is emptied and the new data entries are displayed in table.

Next, add the function that removes the filtering. Once again, make sure that you use your own URL:

```
onClear : function() {
    var serviceURL = this.getView().getModel("Settings").getProperty("/oDataUrl");
    var oModel = new sap.ui.model.odata.v2.ODataModel(serviceURL);
    this.getView().byId("flightsTable").setModel(oModel);
}
```

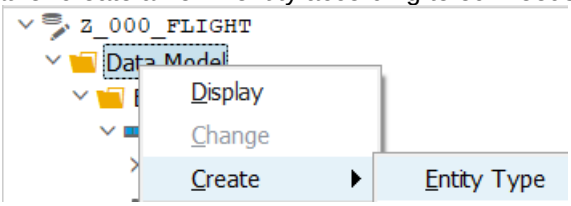
Do not forget to save all your changes and to submit them to the SAP system. When you load the application for the next time, it will look like on the screenshot below. Test the implemented functionality.



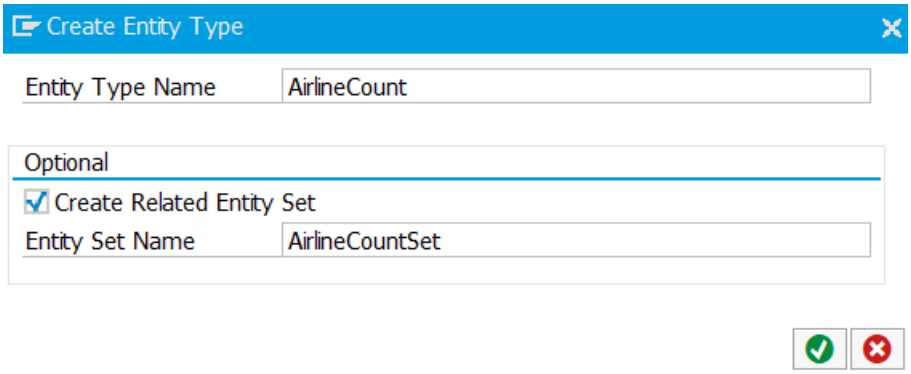
Task 9: Use Function Imports

Short description: The classical OData Interface was only designed to support CRUD operations which are sufficient for most transactional use cases. However, more and more applications are designed to provide nice Charts and analytical dashboards. In such cases it would be rather inefficient to load whole tables from the database to the UI5 application and to aggregate the data in the frontend (we did so for example in task 7). Function imports are an extension for OData to implement customized functionality that do not fit into the classical CRUD schema. In this task we will implement a function import to count the number of flights, but only transmit the final result to our OData application.

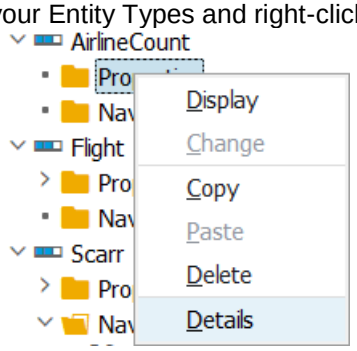
Create a new Entity that will contain the result of our Function Import. Since we only want to count the number of flights per carrier we will not use an existing ABAP Dictionary structure, but rather create an own entity according to our needs.



Enter AirlineCount as the Entity Type and click on "Create Related Entity Set", since our return type will be whole table that contains the counted number of flights for each carrier.



Expand the AirlineCount in your Entity Types and right-click on Properties -> Details



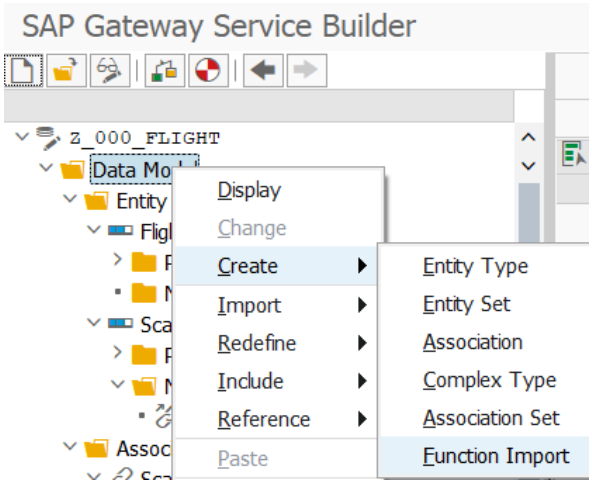
We will now specify which fields / properties our entity shall contain. Our result will only contain two elements, the carrierid (e.g. 'AA') and the counted number of connections (e.g.'22') for this carrier. Therefore, use the button "**Insert Row**" twice to create the new properties and set the datatype and characteristics according to the following picture:

Properties

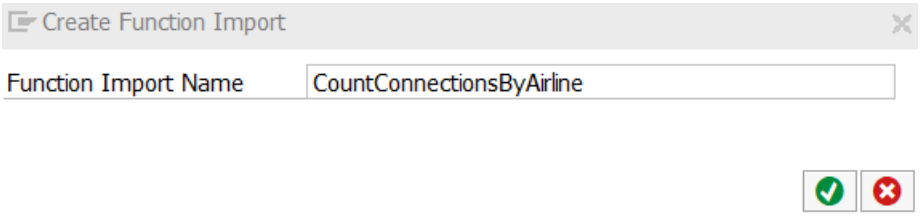
Name	Is Key	Edm Type	Prec.	Scale	Max	Unit Prop.	Creat.	Updat.	Sorta.	Nullab.	Filt.	Labe
CARRID	☑	Edm.String	0	0	0		☑	☑	☑	☐	☑	
COUNT	☐	Edm.Int32	0	0	0		☑	☑	☑	☑	☑	

Save your changes

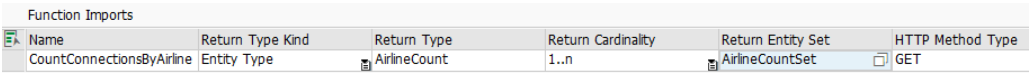
Now we are going to create the function import. Right-click on **"Data Model" -> Create -> Function Import**



Name your Function Import "**CountConnectionsByAirline**"

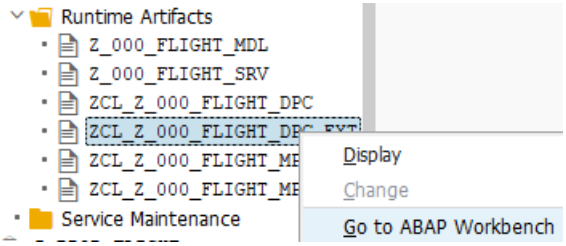


Now, we have to specify the interface of our Function Import. We have to define which Parameters we will return and via which HTTP method we are going to use the import. Of course our return type will be the new entity we just created. The method via which we will call the function Import is via a HTTP GET. Change the characteristics of the function import according to the next picture (**do not forget to change the cardinality** to **1..n**; in case you cannot set AirlineCountSet save your project and try again.):



Save and **generate** your Gateway Project

Our next step is to implement the functionality of our new function import. As always navigate to your class ZCL_Z_000_FLIGHT_DPC_EXT in your runtime artifacts and navigate to the ABAP Workbench.



Add the following code to your **public section** of the **DEFINITION** part of your **DPC_EXT** class. This method is used by all function imports. In order to distinguish between different function import calls you would use CASE or IF branches based on the name of the called function import.

```
METHODS /iwbep/if_mgw_appl_srv_runtime~execute_action REDEFINITION.
```

Implement your function module by using the following code:

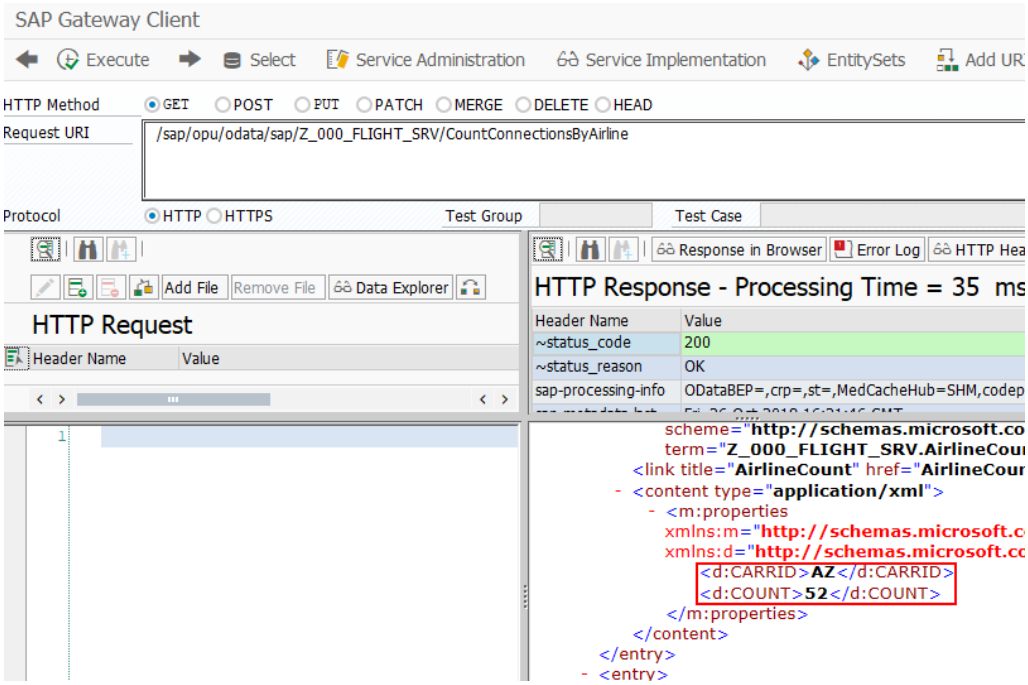
```
CLASS zcl_z_000_flight_dpc_ext IMPLEMENTATION.
  METHOD /iwbep/if_mgw_appl_srv_runtime~execute_action.

    " Implement Function Import 'CountConnectionsByAirline'
    IF iv_action_name = 'CountConnectionsByAirline'.
      TYPES: BEGIN OF sflight_count,
               carrid TYPE sflight-carrid,
               count  TYPE i,
             END OF sflight_count.
      DATA: ls_entity        TYPE zcl_z_000_flight_mpc=>ts_airlinecount,
            lt_entityset     TYPE zcl_z_000_flight_mpc=>tt_airlinecount,
            it_sflight_count TYPE TABLE OF sflight_count,
            wa_sflight_count TYPE sflight_count.

      SELECT carrid COUNT( * ) AS count
      FROM sflight
      INTO CORRESPONDING FIELDS OF TABLE it_sflight_count
      GROUP BY carrid
      ORDER BY carrid.

      LOOP AT it_sflight_count INTO wa_sflight_count.
        ls_entity-carrid = wa_sflight_count-carrid.
        ls_entity-count = wa_sflight_count-count.
        APPEND ls_entity TO lt_entityset.
      ENDLOOP.

      copy_data_to_ref( EXPORTING is_data = lt_entityset
            CHANGING cr_data = er_data ).
    ENDIF.
     " Continue with next Function Import

  ENDMETHOD.
```

Save and **activate** your code.

Now it is time to check if our function module is working. Therefore navigate to transaction **/n/IWFND/MAINT_SERVICE**, **double-click** on your service in the catalog and choose SAP Gateway Client. Use the following Request URI to query your function import (HTTP GET)

```
/sap/opu/odata/sap/Z_$$_###_FLIGHT_SRV/CountConnectionsByAir-
                                        line
```

The result should look like this:



You could now consume this function import in your UI5 application. A typical use case would be to show the distribution of flights for the different carriers in a pie or donut chart. Such analytical / business intelligence applications are the major use cases for function imports.

Challenge: Implement the option to change an existing flight

Challenge 1: Currently, you can only get an overview of all flights or create new flight data. Now, we want to implement a function to change existing flights through a separate view. For this, we first have to implement the **update** method of our service. Go to the service implementation, and open the ABAP Workbench. There, add the method to update an existing flight. The implementation of the method is similar to the **create** method. Make sure to correctly read the key fields of flight to be updated (**carrid, connid**), and modify the entry in the database with an ABAP statement. For simplification reasons, you can assume that all values of the flight entry are updated (except for the key elements). Do not forget to save and activate your service at the end.

Challenge 2: Now, we want to create a second view which shows the flight data to be changed in editable fields. You have to implement the **press** event for the table rows, and pass the values from the table to the second view. In the second view, assign the passed values to input fields. Make sure that the key elements of your flight (carrid, connid) are not editable. Furthermore, create a back button in order to be able to navigate back to the table view. You can refer to the SAPUI5 exercise for further assistance on how to implement the **press** event and pass values between views. Finally, add a button below the input fields. When the button is clicked, it shall call the **update** service you implemented previously, and return a success or error message if your flight was updated correctly. You can refer to the slides belonging to this exercise for further assistance on how to implement this service call in JavaScript.

Challenge 3: By now we have only specified the return parameters and access protocol of our function import. Similar to the CRUD methods a function module can also work with input parameters. Navigate to your SAP Gateway Project and specify an import parameter for your CountConnectionByAirline function import. The parameter should specify the airline for which you would like the now the counted number of flights. Afterwards change your implementation accordingly (this works similar to **Challenge 1**). Finally consume your function import in your UI5 application. Therefore, either create a chart control and consume the number of flights for each airline (original implementation) or consume the number of flights for one airline specified by the user (challenge implementation). You may find useful charts on the following website:

https://sapui5.hana.ondemand.com/#/controls